

- Modify the startup command in the Docker file to include the New Relic agent:

```
CMD ["java", "-javaagent:/newrelic/newrelic.jar", "-jar", "imagecaptioning.jar"]
```

### Step 8: Build and deploy the updated Docker container

- Rebuild the Docker image:

```
docker build -t my-springboot-app
```

- Run the container:

```
docker run -d -p 8080:8080 my-springboot-app
```

- Navigate to the New Relic dashboard and verify that performance metrics are being received.  
Once the backend container is successfully integrated with New Relic, the dashboard provides valuable insights such as:
  - Response times and throughput:** Identifies slow requests and API bottlenecks.
  - Error rates and exceptions:** Tracks application failures and logs stack traces.

- Database query performance:** Monitors slow SQL queries and optimises database calls.
- External service interactions:** Ensures seamless API communication and third-party integration performance.  
We can view all the metrics by navigating through the sections in the left-side panel of the dashboard as shown in Figures 1 and 2.

By leveraging real-time monitoring with the New Relic Java agent, developers can detect and resolve issues faster, optimise backend performance, and ensure a seamless user experience.

## Python agent for model monitoring

The machine learning model, developed using Python, is deployed within a Docker container running a Django application. To ensure optimal performance, the New Relic Python agent was integrated, providing real-time monitoring of model inference times, tracking Python process metrics, identifying bottlenecks, and ensuring the model serves predictions efficiently.

The following steps outline the process of integrating the New Relic Python agent with a Dockerized machine learning model hosted on Django.

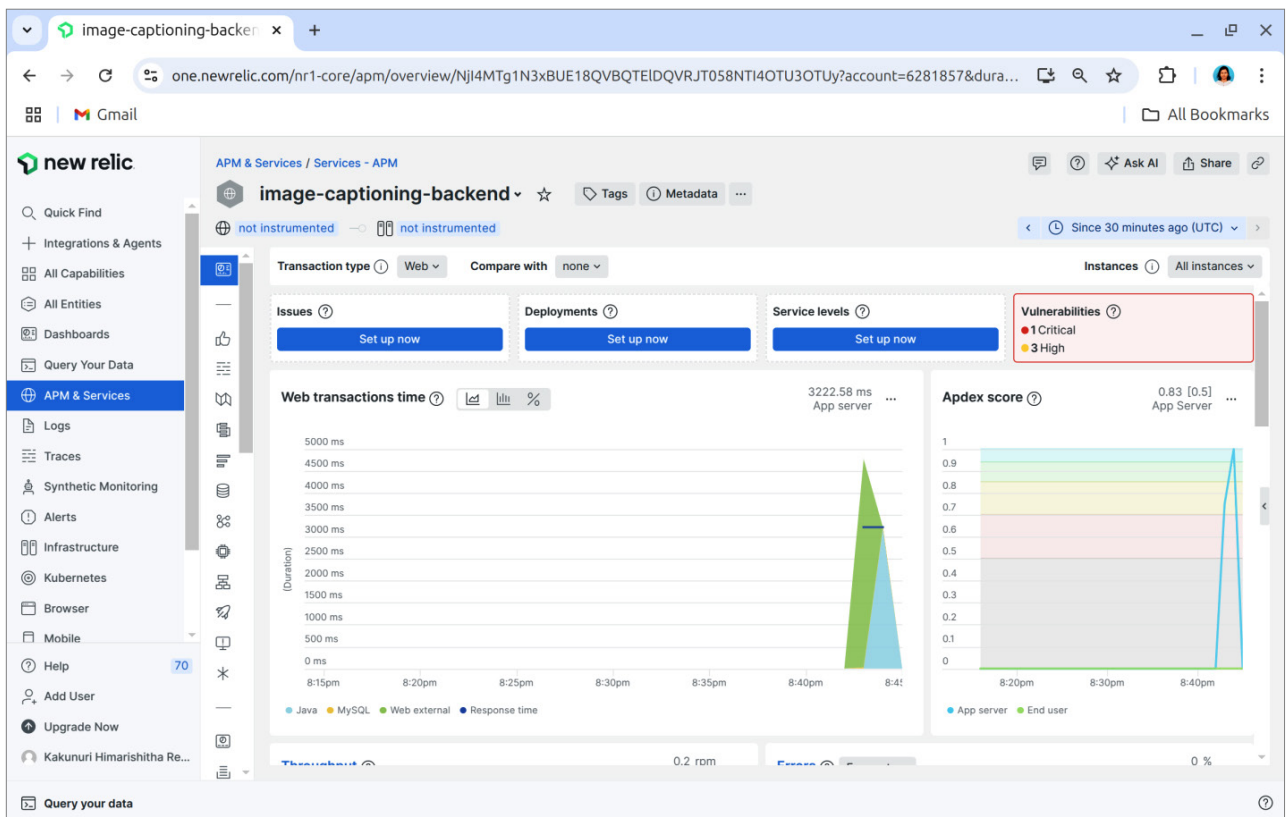


Figure 1: Dashboard showing web transaction times, Apdex score, and detected vulnerabilities for the image-captioning-backend service